GhostQPU explained

How it works, for a team with no quantum computing background

Hackathon project · Quantum Security & Cryptography track · October 10, 2026

The one-minute version

GhostQPU checks whether a quantum computing job really ran on a quantum computer, or was faked on an ordinary computer.

- **The worry:** a middleman who sells quantum computer time could run your job on a free laptop simulator instead, and still charge you the quantum price.
- **Our trick:** we hide about 30 tiny test programs, called **canaries**, inside your job. Real quantum hardware has physical quirks that a simulator leaves out, and the canaries measure them.
- **The result:** on a real IBM quantum computer, GhostQPU told real results from faked ones correctly every time, for about 11 seconds of machine time per check.

You do not need to understand quantum physics to work on this project. The next section gives you just enough to follow the rest.

Quantum computing, just enough

You only need six ideas, and none of them need math.

Idea	What it means	Everyday comparison
Qubit	The basic unit of a quantum computer. When you read it, you get a 0 or a 1.	A spinning coin that lands heads or tails when you look.
Gate	An instruction that changes a qubit. An "X gate" flips it: 0 becomes 1, 1 becomes 0.	Turning the coin over by hand.
Circuit	A small program: a list of gates, then a reading of the qubits at the end.	A recipe.
Shot	One run of a circuit. Answers are random, so we run each circuit many times (we use 1000).	Tossing the coin 1000 times and counting heads.
Noise	Real hardware is imperfect, so it sometimes gives the wrong answer.	A slightly bent coin.
Calibration	A report IBM publishes for each machine, listing how error-prone each qubit and gate is. Updated about once a day.	The manufacturer's spec sheet.

The key fact for this project: today's quantum computers are small enough that a laptop can imitate them. Give the laptop the calibration report, and it produces answers with about the right amount of noise.

We used **ibm_marrakesh**, a real IBM quantum computer with 156 qubits. Anyone can use IBM's machines free for 10 minutes every 28 days.

The problem: a fraud nobody has tested

Quantum computer time is expensive and often sold through resellers. Customers send a job and get results back, but they never see the machine.

That opens a cheap scam, in three steps:

1. A customer sends a job to a dishonest reseller.
2. The reseller downloads IBM's public calibration report and runs the job on a free simulator.
3. The customer receives believable results and pays for quantum time that was never used.

The 2025 research paper our project builds on assumes this is too hard to pull off, and never actually tries it. We built the scam ourselves (the **Forger**), and then a way to catch it (the **Detector**).

> **Analogy:** you pay for a hand-made watch and receive a convincing replica built from the official spec sheet. The spec sheet lists the important numbers, but not every tiny imperfection of the real thing. That gap is how we catch the fake.

The big idea: canaries

A canary is a tiny test circuit whose correct answer we already know. Usually that answer is "this qubit should read 0." Any wrong answer is pure noise from the machine, and that noise is the fingerprint we read.

The name comes from the canary that coal miners carried as an early warning. We shuffle our canaries in among the customer's real circuits, so a reseller cannot tell which circuits are checks. Each canary targets one quirk that real hardware has and the public calibration report leaves out.

1. Repeated flips (errors stack up)

Flip a qubit 128 times. An even number of flips should bring it back to 0. Each real flip is very slightly off, and the small misses add up faster than the report predicts. The simulator treats every miss as small and independent, so its errors pile up slowly.

Measured: real hardware read the wrong answer 32% of the time; the forgery only 4%.

2. Neighbor influence (crosstalk)

Leave one qubit sitting idle, with its neighbor set to 0 in one test and to 1 in another. On real hardware the neighbor nudges the idle qubit, so its result depends on the neighbor. The simulator models every qubit in isolation, so the neighbor makes no difference.

Measured after 16 microseconds: the neighbor shifted the real qubit by 7.8%; the forgery shifted by 0.4%, which is just random noise.

3. Readout in context

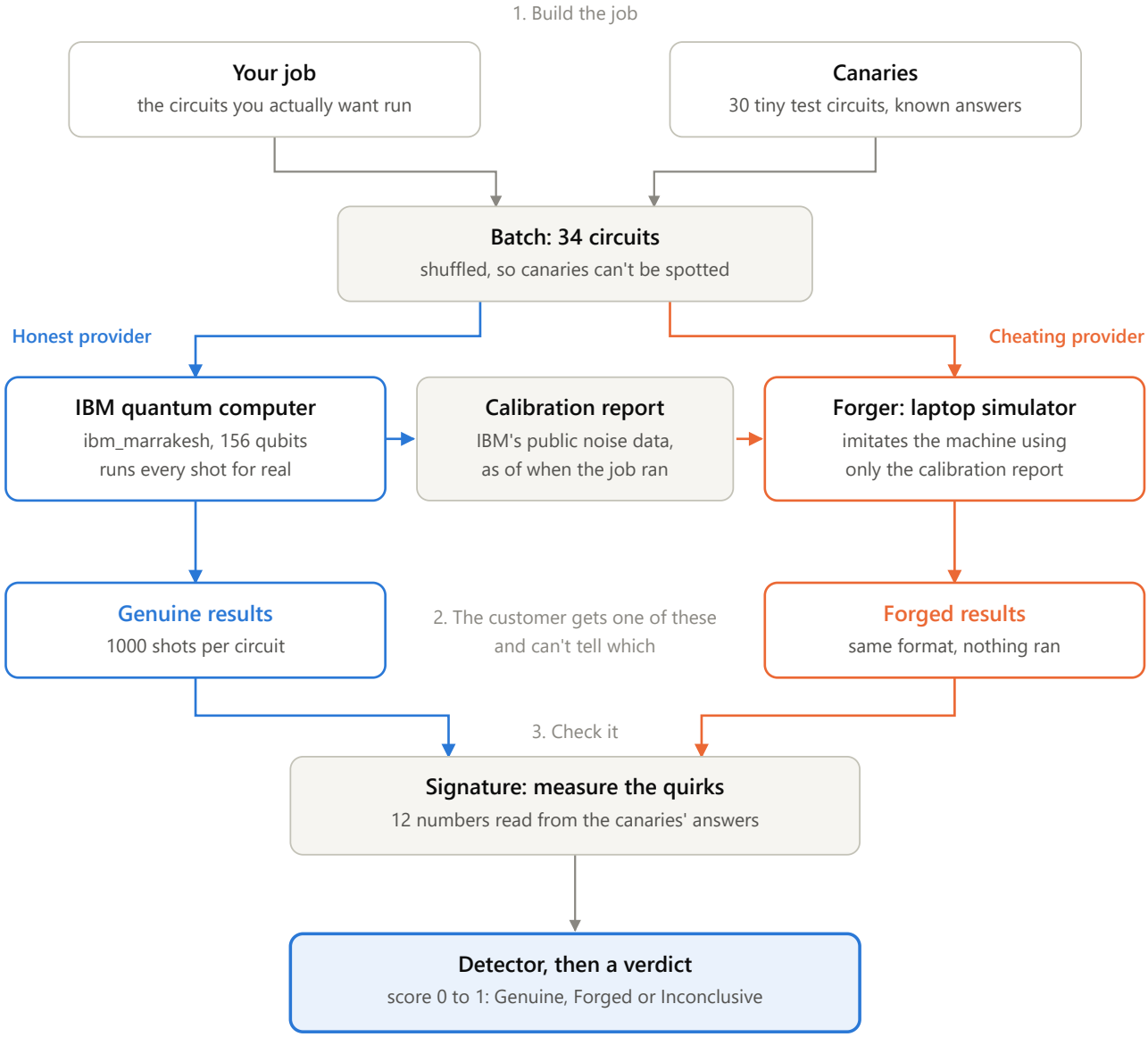
Read a qubit while its neighbor holds a 0 or a 1. Real readout errors are higher than the published numbers and shift with the neighbor. The simulator uses exactly the published numbers.

Measured: real readout error 1.1 to 1.4%; forgery 0.3 to 0.5%.

We also check whether the results **drift over time** during the job. A real machine drifts slightly while it runs; a simulator's results are perfectly even from start to finish.

How the system works

The honest provider and the cheater both return results in exactly the same format. That swap is the attack, and GhostQPU decides which one happened.



Blue is the honest path, orange is the cheat. The Forger only knows what IBM publishes, which is exactly what a real cheater would know.

Step by step

1. **Build the batch.** We pick 4 pairs of well-behaved neighboring qubits from today's calibration report, write 30 canary circuits on them, and shuffle them in with 4 normal circuits.
2. **Submit to IBM.** The batch runs on the real machine. We switch off IBM's built-in error-hiding tricks, because they would hide the very quirks we measure.
3. **Collect.** When the job finishes, we save every shot in order and download the calibration report that was live at the moment the job started.
4. **Forge a twin.** The Forger simulates the exact same circuits from that calibration report. This is the strongest fair cheater: it knows everything public, at the right moment.
5. **Measure.** For both the real and forged results, we compute 12 numbers (the "signature") describing the quirks above.
6. **Decide.** A small, readable model turns those numbers into a score from 0 (genuine) to 1 (forged). Scores between 0.35 and 0.65 are reported as "Inconclusive."

> **Fair testing rule:** a job is never scored by a model that was trained on that job or its twin. Otherwise the model could just memorize the answers.

The code, file by file

Everything lives in the `quantum_hack` folder. Each file has one job.

File	What it does, in plain words
`ghostqpu/snapshot.py`	Saves IBM's calibration report, and turns a saved report back into a simulator. The Forger is built from this.
`ghostqpu/canaries.py`	Picks good qubits, writes the 30 canary circuits, mixes them with normal circuits, and shuffles them into a batch.
`ghostqpu/execution.py`	Sends a batch to IBM and collects the answers (honest path). Also contains the Forger (cheating path).
`ghostqpu/runs.py`	Saves and loads results. Each run is one folder in `data/runs/`.
`ghostqpu/signature.py`	Reads the canaries' answers and computes the 12 quirk numbers.
`ghostqpu/detector.py`	Trains the small model, scores jobs fairly, and gives verdicts.
`ghostqpu/cli.py`	The commands we type: add-account, accounts, submit, collect, evaluate, null-test.
`app.py`	The demo website, with three tabs: Verify a job, Evidence, Detector.
`tests/`	24 automatic checks that run in about a minute, without using any IBM time.
`demo/GhostQPU_demo.mp4`	A 2 minute 23 second captioned demo video.

What we found on real IBM hardware

On two real jobs, the hardware and the forgery looked clearly different on every canary, and the Detector labeled all four results correctly.

Canary	Real hardware	Forger
Wrong answers after 128 flips	32%	4%
Shift caused by a neighbor (after 16 microseconds)	7.8%	0.4%
Readout error	1.1 to 1.4%	0.3 to 0.5%

Job	What it really was	Detector score	Verdict
Run 1	Real IBM hardware	0.02	Genuine
Run 1 twin	Forged	0.98	Forged
Run 2	Real IBM hardware	0.06	Genuine
Run 2 twin	Forged	0.96	Forged

Each job used **11 seconds** of quantum time. With 10 free minutes per account, one account can afford about 50 checks.

> **Sanity check (the "null test").** We also ran the whole system with a simulator playing both sides, so there was nothing real to find. The Detector correctly could not tell them apart (AUC 0.40, about a coin flip). This shows it reacts to real physics, not to some formatting difference in our own code.

How to run it

Open a terminal in the `quantum_hack` folder. On Windows, `python` below means `.venv\Scripts\python`.

Once: connect an IBM account

```
python -m ghostqpu add-account alice     # asks for your IBM API key (hidden)
python -m ghostqpu accounts              # lists machines and free time left
```

Get the API key at quantum.cloud.ibm.com. Never paste it into chats or shared files.

Each check

```
python -m ghostqpu submit --account alice --backend ibm_marrakesh --shots 1000
python -m ghostqpu collect               # saves real results, forges the twin
python -m ghostqpu evaluate              # scores the Detector (needs 2+ real runs)
```

The demo

```
python -m streamlit run app.py           # opens the website at localhost:8501
python -m ghostqpu null-test             # the sanity check
```

What judges might ask

Couldn't a smarter cheater fake the quirks too?

Possibly. Our Forger uses everything IBM publishes, which is what a realistic cheater has. A cheater who knew our canaries could try to add fake quirks; testing that "canary-aware forger" is our next step.

Is two jobs enough to trust the result?

It is enough for a strong early result, not a final one. Both jobs ran on the same day. Next we repeat on other days, after IBM recalibrates, and on other machines.

Doesn't this waste the customer's quantum time?

A check costs about 11 seconds and runs alongside the customer's own circuits. Their results are not affected.

Why not just trust IBM directly?

Many people buy through resellers and cloud marketplaces. The paper we build on names the dishonest reseller as a real threat, but never tested a working cheat.

What if the Detector fails one day?

That is also a result: it would show that public calibration data is enough to fake a quantum computer, which is a security warning in itself.

Glossary

Qubit	The basic unit of a quantum computer; reads as 0 or 1.
Gate	An instruction that changes a qubit. An X gate flips it.
Circuit	A small quantum program: gates, then a reading.
Shot	One run of a circuit. We use 1000 per circuit.
Noise	The machine's mistakes.
Calibration report	IBM's public spec sheet of each machine's noise, updated about daily.
QPU	Quantum processing unit: the quantum computer itself.
QPU seconds	How much quantum machine time a job used, which is what customers pay for.
Provider / reseller	Whoever runs your job and returns results; may be honest or not.
Canary	A hidden test circuit with a known answer that exposes hardware quirks.
Batch	Your circuits plus the canaries, shuffled together and sent as one job.
Forger	Our simulator that fakes results using only public calibration data.
Twin	The forged copy of a real run: same circuits, same calibration.
Genuine / Forged	Ran on real hardware / produced by a simulator.
Signature	The 12 numbers measured from the canaries' answers.
Detector	The small model that turns a signature into a score.
Score	0 means genuine, 1 means forged; 0.35 to 0.65 is "Inconclusive."
Held-out	Scored by a model that never saw that job during training.
AUC	How well the Detector separates the two kinds: 1.0 is perfect, 0.5 is a coin flip.
Null test	A sanity check with simulators on both sides, where the Detector should fail.